

The Globe:

OPOSICIONES

TAI · BLOQUE 3

BLOQUE 3 · TEMA 05 · REPASO

T05 · Repaso del tema

DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS.
ELEMENTOS Y COMPONENTES SOFTWARE. PATRONES
DE DISEÑO Y LENGUAJE DE MODELADO UNIFICADO
(UML)

PREPARACIÓN

Técnicos Auxiliares de Informática

FORMADOR

Pablo Arellano

■ CÓMO SE USA ESTE REPASO

01 Cómo se usa

Este repaso es una síntesis del tema **B3·T05 — Programación Orientada a Objetos**, pensada para el repaso de última hora. Reorganiza lo que ya está en el material de Pablo Arellano —no añade contenido nuevo—: un **mapa** de las seis grandes secciones, los pares de conceptos que más se confunden y una **ficha de cifras** con las listas memorizables. La fuente de verdad sigue siendo el tema en página fiel.

Recorrido sugerido: 1) ubica cada bloque con el *mapa*; 2) repasa «No confundir» justo antes del examen, que es donde caen los distractores; 3) memoriza la *ficha de cifras*. Cada punto resumido aquí está desarrollado en el tema, en la página indicada.

■ ESTRUCTURA DEL TEMA

02 Mapa del tema

LAS 6 GRANDES SECCIONES

- | | |
|---|--|
| 1 | Diseño y POO <small>pág. 3-4</small>
Fases del desarrollo OO (especificación · diseño · codificación · prueba · mantenimiento) y características: abstracción, encapsulamiento, ocultación, polimorfismo y herencia. |
| 2 | Principios generales <small>pág. 5-9</small>
SOLID (5) · DRY · IoC · YAGNI · KISS · Ley de Demeter. |
| 3 | Elementos y componentes <small>pág. 10-15</small>
Clases, objetos, métodos, atributos, mensajes, herencia, composición, polimorfismo y sobrecarga. |
| 4 | Ventajas e inconvenientes <small>pág. 16-17</small>
Reusabilidad, interoperabilidad, mantenimiento, acoplamiento bajo / cohesión alta · frente a curva de aprendizaje y eficiencia de los lenguajes OO puros. |
| 5 | Patrones de diseño <small>pág. 18-39</small>
GoF: creacionales, estructurales y de comportamiento · patrones GRASP (9). |
| 6 | UML <small>pág. 40-68</small>
Diagramas de estructura (vista estática) y de comportamiento (vista dinámica). |

■ PARES QUE SE CONFUNDEN

03 No confundir

Sobrecarga ≠ Sobreescritura

Sobrecargar = varios métodos con el mismo nombre y distinta signatura **dentro de una misma clase**; **no** implica herencia. **Sobrescribir** (@Override) = redefinir en una clase hija un método heredado; **sí** implica herencia. El tipo de retorno no forma parte de la signatura.

Encapsulamiento ≠ Ocultación

Encapsulamiento = reunir en una misma entidad los elementos que le pertenecen (aumenta la cohesión). **Ocultación** = la estructura interna queda oculta y solo se accede a ella a través de los métodos. El propio tema avisa de no confundirlos.

Herencia («es-un») ≠ Composición («tiene-un»)

Herencia: una subclase asume atributos y métodos de la superclase → relación *es-un*. **Composición**: se crean clases complejas agregando objetos de clases existentes → relación *tiene-un*.

Agregación ≠ Composición (UML)

Agregación: rombo **hueco**; relación todo-parte débil (*son-parte-de*). **Composición**: rombo **lleno**; propiedad fuerte, coinciden los tiempos de vida del todo y de las partes.

«usa»/«include» ≠ «extiende»/«extend»

«usa»: comportamiento **común** extraído a un CU compartido. **«extiende»**: comportamiento **opcional**/adicional; la flecha parte del CU adicional hacia el básico (punto de extensión).

Herencia simple ≠ múltiple

Simple: hereda de una sola superclase (p. ej. Java). **Múltiple**: hereda de más de una (p. ej. C++); riesgo de **ambigüedad** y colisiones de métodos con la misma signatura.

D. de Secuencia ≠ D. de Comunicación

Muestran la misma información. **Secuencia**: eje vertical = tiempo, énfasis en el **orden** de los mensajes. **Comunicación** (antes «colaboración»): sin secuencia temporal; numera los mensajes y resalta los **vínculos**.

Clase abstracta ≠ Interfaz

Clase abstracta: tiene al menos un método sin implementar y **no puede instanciarse**. **Interfaz**: especificación de un conjunto de operaciones visibles; UML la dibuja con «Interface» o con un círculo.

■ CIFRAS Y LISTAS MEMORIZABLES

04 Ficha de cifras

LISTAS Y RECUENTOS

5	Principios SOLID S responsabilidad única · O abierto/cerrado · L sustitución de Liskov · I segregación de interfaz · D inversión de dependencia.
6	Principios generales SOLID · DRY · IoC · YAGNI · KISS · Ley de Demeter.
9	Patrones GRASP Experto · Creador · Controlador · Bajo acoplamiento · Alta cohesión · Fabricación pura · Polimorfismo · Indirección · No hable con extraños.
6 · 7 · 11	Patrones GoF por familia Creacionales (6): Abstract Factory, Builder, Factory Method, Prototype, Singleton, Model View Controller. Estructurales (7): Adapter/Wrapper, Bridge, Composite, Decorator, Facade, Flyweight, Proxy. Comportamiento (11): Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.
+ · - · #	Visibilidad UML + público (cualquier clase) · - privado (solo la propia clase) · # protegido (clase y subclases).

UML EN CIFRAS

13	Tipos de diagrama UML 6 de estructura: clases, componentes, objetos, estructura compuesta, despliegue, paquetes. 7 de comportamiento: actividad, casos de uso, máquina de estado y los 4 de interacción (secuencia, comunicación, global de interacción, tiempos).
1995→2017	Hitos de UML Primer borrador 1995 · UML 1.1 aprobado por la OMG en 1997 · UML 2.0 (nuevo estándar) en 2005 · normalización ISO/IEC 19505-1 y -2:2012 · versión actual 2.5.1 (diciembre 2017).
3	Métodos OO precursores Booch (G. Booch) · OMT (J. Rumbaugh) · OOSE (I. Jacobson) → se unen en Rational.